Recueil d'exercices Python

CPES ≪ Sciences des données, arts et cultures ≫ : Introduction à l'algorithmique

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre SENELLART
pierre@senellart.com

12 décembre 2023

1 Manipulation des outils et notions de base de Python3

Exercice 1

Écrivez une fonction Python `degres_en_radians` prenant en paramètre une valeur d'angle en degrés représentée par un nombre à virgule flottante `degres`. La fonction doit retourner la valeur en radians de ce même angle.

Exercice 2

Écrivez une fonction Python `compter_chiffre_dans_nombre` prenant en paramètre une chaîne de caractères `nombre` représentant un nombre décimal ou entier (par exemple, le nombre `145.35` sera représenté par `"145.35"`), ainsi qu'un nombre entier représentant un chiffre `chiffre` (entre 0 et 9 donc). La fonction doit retourner le nombre de fois que `chiffre` apparaît dans `nombre` (`compter_chiffres_dans_nombre("145.35", 5)` doit retourner 2 par exemple).

Exercice 3

Écrivez une fonction Python `compter_tous_les_chiffres_dans_nombre` prenant en paramètre une chaîne de caractères `nombre` représentant un nombre décimal ou entier. Cette fonction fera appel dans son corps à la fonction `compter_chiffre_dans_nombre`, et devra retourner, pour chaque chiffre (donc de 0 à 9), le nombre de fois qu'il apparaît dans `nombre`. On pourra pour cela utiliser un dictionnaire, où chaque clé est un des chiffres, et chaque valeur associée est son nombre d'occurrences.

Exercice 4

Écrivez une fonction Python `inverser_chaine` prenant en paramètre une chaîne de caractères `chaine` et retournant cette même chaîne de caractère inversée : par exemple `"bonjour"` doit devenir `"roujnob"`.

Exercice 5

Écrivez une fonction Python `palindrome_numerique_taille_5` ne prenant pas de paramètre et retournant tous les palindromes numériques de taille 5 possibles. Un palindrome numérique est un entier qui a la même valeur qu'il soit lu de gauche à droite ou de droite à gauche (par exemple, `54354`, `1221` et `2996992`) sont des palindromes numériques.

Exercice 6

Écrivez une fonction Python `verification_palindrome` prenant en entrée une chaîne de caractère `chaine`, qui retourne `True` si la chaîne est un palindrome ou `False` sinon. Dans cet exercice, on ne se limitera pas aux palindromes numériques mais à n'importe quelle chaîne de caractères, contenant donc potentiellement des chiffres, des lettres, ou des caractères spéciaux.

Exercice 7

Écrivez une fonction Python `sous_chaine` prenant en paramètres :
— Une chaîne de caractère `chaine`
— Un nombre entier `debut`
— Un nombre entier `fin`
Cette fonction retournera la sous-chaîne de caractères entre les deux indices `debut` et `fin` tous deux inclus. On s'assurera que la situation est valide, à savoir qu'il est possible de générer une telle sous-chaîne. Si ce n'est pas le cas, on retournera un message indiquant l'invalidité de la situation (par exemple, il n'est pas possible de retourner la sous-chaîne de la chaîne `"algorithmique"` entre les indices 3 et 48).

Exercice 8

Écrivez une fonction Python `remplacement_caractere_dans_chaine` prenant en paramètres (dans cet ordre) :
— une chaîne de caractères `chaine`
— un caractère `a_remplacer`
— un caractère `remplacement`
Cette fonction doit retourner la chaîne `chaine` où l'on aura remplacé toutes les occurences du caractère `a_remplacer` par le caractère `remplacement`. Par exemple, `remplacement_caractere_dans_chaine` donnera `"Le laxgage Pythox"`. Bien entendu, on s'empêchera ici d'utiliser la fonction `replace()` déjà existante sur les chaînes de caractères en Python.

Exercice 9

Écrivez une fonction Python `chaine_compressee` prenant en paramètre une chaîne de caractère `chaine`. La fonction doit retourner une version compressée de cette chaîne, en comptant les occurences successives des caractères. Quelques exemples :
— La chaîne `"aaabbbbcccaa"` donnera `"a3b4c3a2"`
— La chaîne `"inutile"` donnera `"i1n1u1t1i1l1e1"`

Exercice 10

Écrivez une fonction Python `pair_ou_impair` prenant en paramètre un nombre entier `nombre`, et retournant un message décrivant si `nombre` est pair ou impair.

Exercice 11

Écrivez une fonction Python `afficher_puissances_limite` qui prend en paramètres un nombre entier `base` et un nombre entier `valeur_somme_max`. La fonction devra retourner les puissances successives de `base` en partant de 0 (sous la forme d'une chaîne de caractères, ou bien d'une liste), et ce tant que la somme de toutes ces puissances ne dépasse par `valeur_somme_max`.

Exercice 12

Écrivez une fonction Python `date_millionnaire` prenant en paramètres :
— Un nombre entier `annee`
— Un nombre à virgule flottante `taux`, qu'on veut strictement supérieur à 1
— Un nombre à virgule flottante positif `somme_depart`
La fonction devra retourner l'année à partir de laquelle la personne ayant un compte d'épargne démarré à l'année `annee` avec une somme initiale `somme_depart` et un taux annuel de gain `taux` deviendra millionnaire. On fera le choix ici de ne pas résoudre analytiquement le problème, mais de faire une résolution itérative.

Exercice 13

Écrivez une fonction Python `pyramide_de_chiffres` ne prenant aucun paramètre en entrée, et retourner pour chaque chiffre k allant de 1 à 9 inclus, une ligne contenant k fois ce chiffre k (il vous faudra donc retourner une chaîne de caractères).

Exercice 14

Écrivez une fonction Python `factorielle` qui implémenter la fonction mathématique factorielle $\cdot!$ définie par $n! = n \times n-1 \times \cdots \times 1$ pour $n \geqslant 1$, et $0! = 1$. On fera le choix d'implémenter une version itérative de la factorielle.

Exercice 15

Écrivez une fonction Python `approximation_e_ordre_n` qui implémente une valeur approchée de e utilisant la formule $\sum_{i=0}^{n} \frac{1}{i!}$. Vous ferez appel dans cette fonction à la fonction `factorielle` définie plus haut. Cette fonction prendra en paramètre un nombre entier positif ou nul `n`.

Exercice 16

Écrivez une fonction Python `maximum` prenant en paramètre une liste de nombres `liste_nombres`. Cette fonction doit retourner la valeur maximale de cette liste. Vous n'utiliserez aucune fonction avancée déjà existante pour y parvenir (en particulier, aucune fonction qui calcule directement le maximum d'une liste).

Exercice 17

Écrivez une fonction Python `minimum` prenant en paramètre une liste de nombres `liste_nombres`. Cette fonction doit retourner la valeur minimale de cette liste. Vous n'utiliserez aucune fonction avancée déjà existante pour y parvenir (en particulier, aucune fonction qui calcule directement le minimum d'une liste).

Exercice 18

Écrivez une fonction Python `est_premier` prenant en paramètre un nombre entier strictement positif `nombre`. Cette fonction doit retourner la valeur `True` si le nombre entier passé en paramètre est premier, ou `False` sinon. Pour ce faire, vous utiliserez une boucle `while`.

Exercice 19

Écrivez une fonction Python `nombre_sauf_multiples`, prenant en paramètres un entier strictement positif `n` ainsi qu'une liste de chiffres `multiples_interdits`. La fonction devra retourner (par exemple, sous la forme d'une liste) les entiers allant de 1 à `n`, sauf ceux qui sont multiples d'au moins un des chiffres présents dans `multiples_interdits`. Par exemple, si on choisit `n = 12` et `multiples_interdits = [2, 5]`, alors la fonction devra retourner `[1, 3, 7, 9, 11]`. Pour y parvenir, vous devrez utiliser une boucle `while`, et utiliser l'instruction `continue` pour ne pas faire l'affichage des nombres qui ne doivent pas l'être, ainsi que l'instruction `break` pour ne pas continuer à chercher d'autres multiples interdits si l'on en a déjà rencontré un.

2 Manipulation des structures de données de base de Python

Exercice 20

Écrivez une fonction `liste_est_vide` prenant en paramètre une liste Python `liste`, et retournant `True` si celle si est vide, `False` sinon.

Exercice 21

Écrivez une fonction `liste_est_vide_autrement` réalisant la même tâche que celle traitée dans l'exercice 20, **d'une façon différente**.

Exercice 22

Écrivez une fonction `acces_aleatoire` prenant pour paramètres une liste Python `liste` ainsi qu'un entier positif ou nul `indice`. Cette fonction devra retourner la valeur de `liste` à l'indice `indice` si une valeur existe à un tel indice, ou `None` sinon. Quelle est la complexité temporelle asymptotique d'un tel accès aléatoire ?

Exercice 23

Écrivez une fonction `modifier_valeur` prenant pour paramètres :
— Une liste Python `liste`
— Un entier positif ou num `indice`
— Une valeur (quelconque) `nouvelle_valeur`
Cette fonction devra modifier la valeur de `liste` à la position `indice`, en écrasant la valeur déjà existante par `nouvelle_valeur`. Cette fonction **retournera deux valeurs :** la liste `liste` modifiée, et une valeur booléenne (`True` ou `False`) décrivant si oui ou non il a été possible de modifier `liste` à l'indice `indice`. Si l'indice passé en paramètre n'est pas valide, alors on ne modifiera pas la liste `liste`. Quelle est la complexité temporelle asymptotique d'un tel changement de valeur ?

Exercice 24

Écrivez une fonction `ajouter_valeur_a_la_fin` prenant en paramètre une liste Python `liste` et une valeur (quelconque) `nouvelle_valeur`, en utilisant la fonction native des listes `append()`. Quelle est la complexité temporelle asymptotique d'un tel ajout ?

Exercice 25

Écrivez une fonction `ajouter_valeurs_a_la_fin` prenant en paramètre une liste Python `liste` et une liste de valeurs `nouvelles_valeurs`, en utilisant la fonction native des listes `extend()`. Quelle est d'après-vous la complexité temporelle de l'ajout de cette liste de valeurs ? Justifier.

Exercice 26

Écrivez une fonction `longueur_liste` prenant en paramètre une liste Python `liste`, et qui retourne sa longueur. Vous écrirez du code dont la complexité temporelle asymptotique est en $O(1)$.

Exercice 27

Écrivez une fonction `longueur_liste_inefficace` prenant en paramètre une liste Python `liste`, et qui retourne sa longueur. Vous écrirez du code dont la complexité temporelle asymptotique est en $O(n)$.

Exercice 28

Écrivez une fonction `liste_au_cube` prenant en paramètre une liste Python `liste` de nombres à virgules flottantes, et qui retourne `liste` dont tous les éléments ont été mis au cube. Vous écrirez cette fonction **sans compréhension de liste**.

Exerice 29

Même question que l'exercice 28, mais en utilisant cette fois-ci la compréhension de liste.

Exercice 30

Écrivez une fonction `suppression_doublons` prenant en paramètre une liste Python `liste` et retournant cette même liste Python sans doublons. On choisira (arbitrairement) de ne conserver que la première occurrence de chaque élément : ainsi la liste `[2, 6, 2, 4, 8, 8, 7, 6]` deviendra `[2, 6, 4, 8, 7]`.

Exercice 31

Écrivez une fonction `rotation_droite_de_liste` qui prend en paramètre une liste `liste` et un entier `n`. Cette fonction devra effectuer `n` décalages vers la droite des éléments de `liste`. Si jamais `n` était plus grand que la taille de la liste, on ferait alors un décalage modulo `n`. Par exemple, si l'on a la liste `[1, 2, 3, 4, 5, 6]` et `n = 8`, alors on aurait un décalage réel `n_reel = 2`, et la fonction retournera alors la liste `[5, 6, 1, 2, 3, 4]`.

Exercice 32

Quelles est la différence principale entre les tuples et les listes en Python ?

Exercice 33

Écrivez une fonction `longueur_tuple` retournant la longueur d'un tuple passé en paramètre `tup`, dont le corps de la fonction ne fait qu'une seule ligne. Quelle est la complexité temporelle asymptotique associée ?

Exercice 34

Écrivez une fonction `indicage_tuples` prenant en paramètre une liste de tuples `liste_tuples`. Cette fonction devra retourner une nouvelle liste de tuples, où chaque tuple contiendra un élément supplémentaire, qui correspondra à son indice dans la liste. Par exemple, [(2, 3), (4), (9, 8, 7), (6, 6, 4, 2)] deviendra [(0, 2, 3), (1, 4), (2, 9, 8, 7), (3, 6, 6, 4, 2)].

Exercice 35

Écrivez une fonction `min_et_max_tuples_taille_3` prenant en paramètre une liste de tuples `liste_tuples`. Chaque tuple sera composé de 3 nombres à virgule flottante. Cette fonction retournera 2 valeurs : le tuple dont la somme des 3 éléments est la plus petite, et le tuples dont la somme des 2 éléments est la plus grande.

Exercice 36

Écrivez une fonction `tuple_trie_deuxieme_element` prenant en paramètre une liste de tuples `liste_tuples`. Chaque tuple sera composé de 2 nombres à virgule flottante. Cette fonction devra retourner une nouvelle liste de tuples triée en fonction de la valeur de leur deuxième élément par ordre croissant (le choix de la méthode de tri est laissé libre).

Exercice 37

Quelle est la principale différence entre un ensemble et une liste en Python ?

Exercice 38

Écrivez une fonction `ajout_element_si_absent` prenant en paramètres un ensemble Python `ensemble` et une valeur quelconque `valeur`. Cette fonction devra ajouter `valeur` à `ensemble` si `valeur` n'est pas déjà dans `ensemble`. Cette fonction retournera la nouvelle version de `ensemble`, et vous devrez utiliser en son sein la fonction native sur les ensembles `add()`. Quelle est la complexité temporelle asymptotique de cette fonction ? Justifier.

Exercice 39

Écrivez une fonction `union_deux_ensembles` qui renvoie l'union de deux ensembles Python passés en paramètres `ensemble_1` et `ensemble_2`, en utilisant la fonction native sur les ensembles `union()`. Quelle est la complexité temporelle asymptotique d'une telle union ?

Exercice 40

Écrivez une fonction `intersection_deux_ensembles` qui renvoie l'intersection de deux ensembles Python passés en paramètres `ensemble_1` et `ensemble_2`, en utilisant la fonction native sur les ensembles `intersection()`. Quelle est la complexité temporelle asymptotique d'une telle intersection ?

Exercice 41

Écrivez une fonction `ensembles_disjoints`, prenant pour paramètres deux ensembles Python `ensemble_1` et `ensemble_2`. Cette fonction renverra `True` si les deux ensembles sont *disjoints*, et `False` sinon. Vous devrez pour cela utiliser la fonction `intersection_deux_ensembles` écrite précédemment. *Rappel : Deux ensembles sont disjoints si et seulement si ils ne partagent aucun élément en commun.*

Exercice 42

Quels sont les deux éléments que l'on retrouve pour chaque entrée dans un dictionnaire ?

Exercice 43

Écrivez une fonction `remplacer_valeur_dictionnaire` prenant en paramètres un dictionnaire Python `dictionnaire`, une clé `cle` et une valeur `nouvelle_valeur`. Cette fonction renverra le dictionnaire dont la valeur à la clé `cle` a été modifiée par la valeur `nouvelle_valeur`. Si jamais `cle` n'est pas dans le dictionnaire, alors ajoutez cette clé et associez-la à la valeur `nouvelle_valeur`. Quelle est la complexité temporelle asymptotique d'une telle fonction ? Justifier.

Exercice 44

Écrivez une fonction `compter_occurrences_dans_liste`, prenant en paramètre une liste Python `liste_elements`. Cette fonction renverra un dictionnaire où chaque clé correspondra à chaque élément distinct de la liste `liste_elements`, chacune associée au nombre d'occurrences de cet élément.

Exercice 45

Imaginez que vous ayez une liste de dictionnaires représentant les ventes d'un magasin. Chaque dictionnaire représente une vente, avec les clés suivantes :

— `'produit'`, le nom du produit
— `'quantite_produit'`, le nombre d'articles vendus
— `'prix_unitaire'`, le prix unitaire de chaque article

1. Écrivez une fonction `montant_total_ventes` retournant le montant total des ventes.
2. Écrivez une fonction `produit_le_plus_vendu` retournant le nom du produit le plus vendu.
3. Écrivez une fonction `quantite_chaque_produit` retournant un dictionnaire Python ayant pour clés le nom des produit, et pour valeurs le nombre d'unités vendues associé.

Exercice 46

Pour chacun des problèmes suivant, indiquer s'il vaut mieux utiliser une liste, un tuple, un dictionnaire ou bien une composition de ces éléments (et préciser cette composition).

— Représenter les coordonnées GPS d'une position sur une carte.
— Représenter une séries de points dans le plan euclidien.
— Représenter plusieurs grandeurs physiques et leur dimension.
— Représenter une classe d'étudiants avec, pour chacun d'entre eux, leur moyenne dans chaque matière.

3 Pratique de la récursivité en Python

A venir.

4 Pratique de la programmation orientée objet en Python

A venir.